COSC 423/523: Introduction to Artificial Intelligence

PA3: Reinforcement Learning

Fall 2026 | 10% of course grade | Due: See Canvas

Introduction

Implement value iteration and Q-learning. Test your agents first in Gridworld, then apply them to a simulated robot controller (Crawler) and Pacman. You will study planning with a known Markov decision process (MDP), learning from interaction, exploration, and generalization through features.

Use Python 3 and run commands from the project folder. In the commands below, `python` must refer to Python 3; use `python3` if required by your installation. The starter code is distributed separately through the course.

Starter folder: `Student_Starter_Code/reinforcement`

Course policies and collaboration

- **COSC 423:** Work individually or in a team of at most three students. If you form a team, you must follow the team-submission rules at the end of this handout.
- **COSC 523:** Complete the assignment individually. Team submissions are not permitted.
- Do not share code across teams or independent submissions. Submitted code must be your own work or the work of your registered, eligible team. Submissions may be checked for inappropriate code similarity.
- Every student must be able to explain the submitted implementation, including every member of a COSC 423 team.
- Do not rename required files or change provided function or class names or signatures. Edit only the files listed below. The engine, layouts, utilities, and public tests are support code.
- Code is autograded for technical correctness. Implementation correctness is the final basis for grading, and submissions may be reviewed individually when necessary.
- No separate report or presentation is required for the regular assignment grade. Contact course staff through the course support channels when you need help. Do not post spoilers or solution code publicly.

Files and local testing

Student files	Purpose
`valueIterationAgents.py`	Batch value iteration.
`qlearningAgents.py`	Tabular and approximate Q-learning agents.
`analysis.py` (optional)	Ungraded parameter-analysis practice; not required for submission.

Useful reference files include `mdp.py` (MDP interface), `learningAgents.py` (base agent classes), `util.py` (including `Counter` and `PriorityQueue`), `gridworld.py` (Gridworld), and `featureExtractors.py` (state-action features).

Supporting files include `environment.py`, `graphicsGridworldDisplay.py`, `graphicsUtils.py`, `textGridworldDisplay.py`, `crawler.py`, `graphicsCrawlerDisplay.py`, `autograder.py`, `testParser.py`, `testClasses.py`, `test_cases/`, and `reinforcementTestClasses.py`. You may run these files but should not edit them.

Run all tests, a single question, or a single test:

```
python autograder.py
python autograder.py -q q2
python autograder.py -t test_cases/q2/1-bridge-grid
```

Assignment scope and grading

Both COSC 423 and COSC 523 follow the same technical requirements and grading allocation. The required work totals 100 PA3 points and is worth 10% of the course grade.

Question	Topic	PA3 points	Status
1	Value iteration	35	Required
2	Bridge crossing analysis	–	Ungraded practice
3	Policies	–	Ungraded practice
4	Tabular Q-learning	35	Required
5	Epsilon-greedy selection	20	Required
6	Bridge crossing revisited	–	Ungraded practice
7	Tabular Q-learning in Pacman	10	Required
8	Approximate Q-learning	Up to 10	Optional bonus
	Required total	100	

Autograder note. The supplied course autograder reports 15 legacy points for the required questions. The course uses the 100-point allocation above. Its default command runs only the Fall 2026 required questions. Run optional questions explicitly by question number.

```
python autograder.py --no-graphics
```

Optional showcase. Up to ten strong teams may be invited to a voluntary 3–5 minute showcase. Selection considers correctness, code clarity, optional Q8 design, and reproducible performance; no separate leaderboard entry is required. An invited team that chooses to present may earn up to +5 percentage points on the final course grade under the syllabus bonus policy. Declining has no penalty. This does not change the individual-work requirement for COSC 523.

MDPs: getting started with Gridworld

Run manual control with the arrow keys, inspect the available options, and try the default random agent:

```
python gridworld.py -m
python gridworld.py -h
python gridworld.py -g MazeGrid
```

The blue dot represents the agent. By default, an intended northward move succeeds only 80% of the time. Gridworld first enters a pre-terminal state (a double box in the GUI), then takes the special `exit` action to reach the true terminal state, `TERMINAL_STATE`, which is not drawn. Returns are discounted; the default discount is 0.9, and `-d` changes it.

Console output describes each transition. Use `-t` for text mode or `-q` to suppress transition output. Positions use Cartesian (x, y) coordinates; arrays are indexed by $[x][y]$, with north increasing y. Most transitions receive zero reward by default; `-r` changes the living reward. If the GUI renders too small, adjust `--windowSize` (`-w`), for example:

```
python gridworld.py -m -w 100
```

Question 1. Value Iteration (35 PA3 points)

Complete `ValueIterationAgent` in `valueIterationAgents.py`. This agent is an offline planner: its constructor receives an MDP and runs the requested number of value-iteration steps before returning. The `-i` option controls the number of planning iterations.

$$V_{k+1}(s) = \max_a \sum_{s'} T(s, a, s') \big[R(s, a, s') + \gamma V_k(s') \big].$$

Implement `runValueIteration`, `computeActionFromValues(state)`, and `computeQValueFromValues(state, action)`. The latter two methods use the current value function in `self.values` to return a greedy action and a state-action Q-value, respectively.

Use batch value iteration. Every value in an iteration must be computed from the same fixed vector from the preceding iteration. Do not update the vector in place and then use those newly updated values for other states within that iteration.

Values V_k reflect the next k rewards. A Q-value or greedy policy computed from V_k includes one more reward, giving Q_{k+1} and the synthesized policy π_{k+1}. Return that synthesized policy. Handle states with no legal actions: their future value is zero and the policy action is `None`.

You may use `util.Counter`, which defaults to zero for missing keys. Be careful with `argMax`: a legal action absent from the counter may still be optimal. In the GUI, squares show values, square quarters show Q-values, and arrows show policies.

```
python autograder.py -q q1
python gridworld.py -a value -i 100 -k 10
python gridworld.py -a value -i 5
```

The second command computes a policy and executes it ten times. Press a key to cycle through values, Q-values, and simulation. Compare the start-state value with the empirical average return. The last command is a short planning run on the default BookGrid.

Grading: Values, Q-values, and policies are checked on a new grid after fixed iteration counts and at convergence (for example, after 100 iterations).

Question 2. Bridge Crossing Analysis (ungraded practice)

BridgeGrid is a grid world map with a low-reward terminal state and a high-reward terminal state separated by a narrow "bridge", on either side of which is a chasm of high negative reward. The agent starts near the low-reward state. With the default discount of 0.9 and the default noise of 0.2, the optimal policy does not cross the bridge. Change only ONE of the discount and noise parameters so that the optimal policy causes the agent to attempt to cross the bridge. Put your answer in question2() of analysis.py. (Noise refers to how often an agent ends up in an unintended successor state when they perform an action.) The default corresponds to:

```
python gridworld.py -a value -i 100 -g BridgeGrid --discount 0.9 --noise 0.2
```

Practice check (ungraded): Verify that you only changed one of the given parameters, and that with this change, a correct value iteration agent should cross the bridge. To check your answer, run the autograder:

```
python autograder.py -q q2
```

Question 3. Policies (ungraded practice)

Consider the DiscountGrid layout. This grid has two terminal states with positive payoff (in the middle row), a close exit with payoff +1 and a distant exit with payoff +10. The bottom row of the grid consists of terminal states with negative payoff (shown in red); each state in this "cliff" region has payoff -10. The starting state is the yellow square. We distinguish between two types of paths: (1) paths that "risk the cliff" and travel near the bottom row of the grid; these paths are shorter but risk earning a large negative payoff. (2) paths that "avoid the cliff" and travel along the top edge of the grid. These paths are longer but are less likely to incur huge negative payoffs.

In this question, you will choose settings of the discount, noise, and living reward parameters for this MDP to produce optimal policies of several different types. Your setting of the parameter values for each part should have the property that, if your agent followed its optimal policy without being subject to any noise, it would exhibit the given behavior. If a particular behavior is not achieved for any setting of the parameters, assert that the policy is impossible by returning the string 'NOT POSSIBLE'.

Here are the optimal policy types you should attempt to produce:

(a) Prefer the close exit (+1), risking the cliff (-10)

(b) Prefer the close exit (+1), but avoiding the cliff (-10)

(c) Prefer the distant exit (+10), risking the cliff (-10)

(d) Prefer the distant exit (+10), avoiding the cliff (-10)

(e) Avoid both exits and the cliff (so an episode should never terminate)

To see what behavior a set of numbers ends up in, run the following command to see a GUI:

```
python gridworld.py -g DiscountGrid -a value --discount [YOUR_DISCOUNT] --noise [YOUR_NOISE]
    --livingReward [YOUR_LIVING_REWARD]
```

To check your answers, run the autograder:

```
python autograder.py -q q3
```

question3a() through question3e() should each return a 3-item tuple of (discount, noise, living reward) in analysis.py.

Note: You can check your policies in the GUI. For example, using a correct answer to 3(a), the arrow in (0,1) should point east, the arrow in (1,1) should also point east, and the arrow in (2,1) should point north.

Note: On some machines you may not see an arrow. In this case, press a button on the keyboard to switch to qValue display, and mentally calculate the policy by taking the arg max of the available qValues for each state.

Practice check (ungraded): Verify that the desired policy is returned in each case.

Question 4. Tabular Q-Learning (35 PA3 points)

Note that your value iteration agent does not actually learn from experience. Rather, it ponders its MDP model to arrive at a complete policy before ever interacting with a real environment. When it does interact with the environment, it simply follows the precomputed policy (e.g. it becomes a reflex agent). This distinction may be subtle in a simulated environment like a Gridword, but it's very important in the real world, where the real MDP is not available.

You will now write a Q-learning agent, which does very little on construction, but instead learns by trial and error from interactions with the environment through its update(state, action, nextState, reward) method. A stub of a Q-learner is specified in QLearningAgent in qlearningAgents.py, and you can select it with the option '-a q'. For this question, you must implement the update, computeValueFromQValues, getQValue, and computeActionFromQValues methods.

Note: For computeActionFromQValues, you should break ties randomly for better behavior. The random.choice() function will help. In a particular state, actions that your agent hasn't seen before still have a Q-value, specifically a Q-value of zero, and if all of the actions that your agent has seen before have a negative Q-value, an unseen action may be optimal.

Important: Make sure that in your computeValueFromQValues and computeActionFromQValues functions, you only access Q values by calling getQValue. This abstraction will be useful for Question 8 when you override getQValue to use features of state-action pairs rather than state-action pairs directly.

With the Q-learning update in place, you can watch your Q-learner learn under manual control, using the keyboard:

```
python gridworld.py -a q -k 5 -m
```

Recall that -k will control the number of episodes your agent gets to learn. Watch how the agent learns about the state it was just in, not the one it moves to, and "leaves learning in its wake." Hint: to help with debugging, you can turn off noise by using the –noise 0.0 parameter (though this obviously makes Q-learning less interesting). If you manually steer Pacman north and then east along the optimal path for four episodes, inspect how the Q-values change.

Grading: We will run your Q-learning agent and check that it learns the same Q-values and policy as our reference implementation when each is presented with the same set of examples. To grade your implementation, run the autograder:

```
python autograder.py -q q4
```

Question 5. Epsilon-Greedy Action Selection (20 PA3 points)

Complete your Q-learning agent by implementing epsilon-greedy action selection in getAction, meaning it chooses random actions an epsilon fraction of the time, and follows its current best Q-values otherwise. Note that choosing a random action may result in choosing the best action - that is, you should not choose a random sub-optimal action, but rather any random legal action.

You can choose an element from a list uniformly at random by calling the random.choice function. You can simulate a binary variable with probability p of success by using util.flipCoin(p), which returns True with probability p and False with probability 1-p.

After implementing the getAction method, observe the following behavior of the agent in GridWorld (with epsilon = 0.3).

```
python gridworld.py -a q -k 100
```

Your final Q-values should resemble those of your value iteration agent, especially along well-traveled paths. However, your average returns will be lower than the Q-values predict because of the random actions and the initial learning phase.

You can also observe the following simulations for different epsilon values. Does that behavior of the agent match what you expect?

```
python gridworld.py -a q -k 100 --noise 0.0 -e 0.1
```

```
python gridworld.py -a q -k 100 --noise 0.0 -e 0.9
```

To test your implementation, run the autograder:

```
python autograder.py -q q5
```

With no additional code, you should now be able to run a Q-learning crawler robot:

```
python crawler.py
```

If this doesn't work, you've probably written some code too specific to the GridWorld problem and you should make it more general to all MDPs.

This will invoke the crawling robot from class using your Q-learner. Play around with the various learning parameters to see how they affect the agent's policies and actions. Note that the step delay is a parameter of the simulation, whereas the learning rate and epsilon are parameters of your learning algorithm, and the discount factor is a property of the environment.

Question 6. Bridge Crossing Revisited (ungraded practice)

First, train a completely random Q-learner with the default learning rate on the noiseless BridgeGrid for 50 episodes and observe whether it finds the optimal policy.

```
python gridworld.py -a q -k 50 -n 0 -g BridgeGrid -e 1
```

Now try the same experiment with an epsilon of 0. Is there an epsilon and a learning rate for which it is highly likely (greater than 99%) that the optimal policy will be learned after 50 iterations? The bridge-crossing response function provided in analysis.py should return EITHER a 2-item tuple of (epsilon, learning rate) OR the string 'NOT POSSIBLE' if there is none. Epsilon is controlled by -e, learning rate by -l.

Note: Your response should not depend on the exact tie-breaking mechanism used to choose actions. This means your answer should be correct even if for instance we rotated the entire bridge grid world 90 degrees.

To check this ungraded practice answer, run the autograder:

```
python autograder.py -q q6
```

Question 7. Tabular Q-Learning in Pacman (10 PA3 points)

Time to play some Pacman! Pacman will play games in two phases. In the first phase, training, Pacman will begin to learn about the values of positions and actions. Because it takes a very long time to learn accurate Q-values even for tiny grids, Pacman's training games run in quiet mode by default, with no GUI (or console) display. Once Pacman's training is complete, he will enter testing mode. When testing, Pacman's self.epsilon and self.alpha will be set to 0.0, effectively stopping Q-learning and disabling exploration, in order to allow Pacman to exploit his learned policy. Test games are shown in the GUI by default. Without any code changes you should be able to run Q-learning Pacman for very tiny grids as follows:

```
python pacman.py -p PacmanQAgent -x 2000 -n 2010 -l smallGrid
```

Note that PacmanQAgent is already defined for you in terms of the QLearningAgent you've already written. PacmanQAgent is only different in that it has default learning parameters that are more effective for the Pacman problem (epsilon=0.05, alpha=0.2, gamma=0.8). This question checks that your completed tabular agent transfers from Gridworld to Pacman and executes correctly. Use the course autograder to evaluate the required implementation.

Hint: If your QLearningAgent works for gridworld.py and crawler.py but does not seem to be learning a good policy for Pacman on smallGrid, it may be because your getAction and/or computeActionFromQValues methods do not in some cases properly consider unseen actions. In particular, because unseen actions have by definition a Q-value of zero, if all of the actions that have been seen have negative Q-values, an unseen action may be optimal. Beware of the argMax function from util.Counter!

To grade your answer, run:

```
python autograder.py -q q7
```

Note: If you want to experiment with learning parameters, you can use the option -a, for example -a epsilon=0.1,alpha=0.3,gamma=0.7. These values will then be accessible as self.epsilon, self.gamma and self.alpha inside the agent.

Note: While a total of 2010 games will be played, the first 2000 games will not be displayed because of

the option -x 2000, which designates the first 2000 games for training (no output). Thus, you will only see Pacman play the last 10 of these games. The number of training games is also passed to your agent as the option numTraining.

Note: If you want to watch 10 training games to see what's going on, use the command:

```
python pacman.py -p PacmanQAgent -n 10 -l smallGrid -a numTraining=10
```

During training, you will see output every 100 games with statistics about how Pacman is faring. Epsilon is positive during training, so Pacman will play poorly even after having learned a good policy: this is because he occasionally makes a random exploratory move into a ghost. As a benchmark, it should take between 1000 and 1400 games before Pacman's rewards for a 100 episode segment becomes positive, reflecting that he's started winning more than losing. By the end of training, it should remain positive and be fairly high (between 100 and 350).

Make sure you understand what is happening here: the MDP state is the exact board configuration facing Pacman, with the now complex transitions describing an entire ply of change to that state. The intermediate game configurations in which Pacman has moved but the ghosts have not replied are not MDP states, but are bundled in to the transitions.

Once Pacman is done training, he should win very reliably in test games (at least 90% of the time), since now he is exploiting his learned policy.

However, you will find that training the same agent on the seemingly simple mediumGrid does not work well. In our implementation, Pacman's average training rewards remain negative throughout training. At test time, he plays badly, probably losing all of his test games. Training will also take a long time, despite its ineffectiveness.

Pacman fails to win on larger layouts because each board configuration is a separate state with separate Q-values. He has no way to generalize that running into a ghost is bad for all positions. Obviously, this approach will not scale.

Question 8. Approximate Q-Learning (up to 10 bonus PA3 points)

Implement an approximate Q-learning agent that learns weights for features of states, where many states might share the same features. Write your implementation in ApproximateQAgent class in qlearningAgents.py, which is a subclass of PacmanQAgent.

Approximate Q-learning uses a feature vector $f(s, a)$ and a dictionary of weights w_i. The provided feature extractors return a `util.Counter` containing nonzero feature values; omitted features have value zero.

$$Q(s, a) = \sum_i f_i(s, a) w_i,$$

$$\delta = r + \gamma \max_{a'} Q(s', a') - Q(s, a), \qquad w_i \leftarrow w_i + \alpha \delta f_i(s, a).$$

Here r is the experienced reward, and δ is the same temporal-difference error used in tabular Q-learning. Use zero for the future-value term when the next state has no legal actions.

By default, ApproximateQAgent uses the IdentityExtractor, which assigns a single feature to every (state,action) pair. With this feature extractor, your approximate Q-learning agent should work identically to PacmanQAgent. You can test this with the following command:

```
python pacman.py -p ApproximateQAgent -x 2000 -n 2010 -l smallGrid
```

Important: ApproximateQAgent is a subclass of QLearningAgent, and it therefore shares several methods like getAction. Make sure that your methods in QLearningAgent call getQValue instead of accessing

Q-values directly, so that when you override getQValue in your approximate agent, the new approximate q-values are used to compute actions.

Once you're confident that your approximate learner works correctly with the identity features, run your approximate Q-learning agent with our custom feature extractor, which can learn to win with ease:

```
python pacman.py -p ApproximateQAgent -a extractor=SimpleExtractor -x 50 -n 60 -l mediumGrid
```

Even much larger layouts should be no problem for your ApproximateQAgent (warning: this may take a few minutes to train):

```
python pacman.py -p ApproximateQAgent -a extractor=SimpleExtractor -x 50 -n 60 -l mediumClassic
```

If you have no errors, your approximate Q-learning agent should win almost every time with these simple features, even with only 50 training games.

Grading: We will run your approximate Q-learning agent and check that it learns the same Q-values and feature weights as our reference implementation when each is presented with the same set of examples. To grade your implementation, run the autograder:

```
python autograder.py -q q8
```

Submission: follow the PA1 workflow

Required code files: `valueIterationAgents.py` and `qlearningAgents.py`. Upload these edited files through the PA3 Gradescope assignment. `analysis.py` is for ungraded practice and is not required for the Fall 2026 graded scope. Do not submit the entire engine unless Gradescope explicitly requests additional files. Keep all provided names and interfaces unchanged.

COSC 423: working individually

1. Submit your own two required Python files to Gradescope as an individual submission.
2. Submit a contribution statement in Canvas stating that you completed the assignment individually.

COSC 423: working in a team of two or three

1. Make **one Gradescope code submission for the team**. Use Gradescope's group-submission feature to select **all team members**. A team must complete this group-submission process.
2. Submit **one team contribution statement through Canvas**, identifying all members and explaining how the work was divided.
3. Every member must be able to explain the team's full submitted implementation.

COSC 523: individual work is required

1. Implement and submit your own two required Python files to Gradescope. Do not join a group submission.
2. Submit your Canvas contribution statement stating that the assignment was completed individually.

Before submitting: Run the required-scope autograder from a clean copy of the assigned starter folder with your edited files. Verify that you uploaded the final versions and registered all eligible team members if applicable. Check Canvas for the due date and assignment links.